

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: XỬ LÝ ẢNH SỐ

PHÂN ĐOẠN TỪNG ĐỐI TƯỢNG BẰNG DEEP LEARNING VỚI STARDIST

Đối chiếu Fiji/ImageJ và cài đặt Python

Mã nhóm: **GR03**

Thành viên: Nguyễn Hồng Quang – B23DCVT361

Nguyễn Trọng Nam Khánh – B23DCCE052

Vũ Dũng – B23DCVT110

HÀ NỘI

Tóm tắt

Phân đoạn từng đối tượng (instance segmentation) là bước nền tảng để đếm và đo hình thái tế bào. Báo cáo nghiên cứu StarDist 2D, một phương pháp dùng mạng CNN để dự đoán xác suất tâm đối tượng và khoảng cách theo các tia, sau đó giải mã đa giác star-convex và loại proposal trùng bằng Non-Maximum Suppression (NMS). Nhóm xây dựng pipeline Python tái lập được, tự cài đặt các thao tác hình học và đánh giá instance, đồng thời dùng plugin StarDist của Fiji/ImageJ làm nền tảng tham chiếu. Otsu và marker-controlled Watershed là hai baseline. Kết quả cuối được đánh giá bằng matching một-một theo IoU, AP tại nhiều ngưỡng, Panoptic Quality, Dice foreground, sai số đếm và thời gian xử lý. Các label TIFF Fiji/ImageJ, log tham số và metric của ba ca kiểm chứng được lưu cùng mã nguồn.

Trên 50 ảnh test DSB2018, StarDist Python đạt AP@0.50 0.846 ± 0.132 và PQ@0.50 0.782 ± 0.117, cao hơn hai baseline cổ điển trong protocol đã công bố. Ba ca low/median/high chạy thật trên Fiji/ImageJ được giữ riêng để đối chiếu liên nền tảng, tránh diễn giải nhầm chúng như benchmark $n = 50$.

Từ khóa: StarDist, instance segmentation, U-Net, star-convex, Fiji, ImageJ, Otsu, Watershed, Panoptic Quality.

Mục lục

Tóm tắt i

Danh mục hình vẽ iv

Danh mục bảng v

1 Giới thiệu 1
- 1.1 Bối cảnh và động cơ . . . 1
- 1.2 Mục tiêu và đóng góp . . . 1
- 1.3 Câu hỏi nghiên cứu . . . 1

2 Cơ sở lý thuyết 2
- 2.1 Bài toán instance segmentation . . . 2
  - 2.1.1 Tiền xử lý và quy ước số học . . . 2
- 2.2 CNN và U-Net . . . 3
  - 2.2.1 Tích chập, receptive field và phi tuyến . . . 3
  - 2.2.2 U-Net làm backbone đa đầu ra . . . 3
- 2.3 Probability map và star-convex polygon . . . 3
  - 2.3.1 Giả thiết star-convex và target radial distance . . . 3
  - 2.3.2 Khả năng biểu diễn và giới hạn . . . 4
- 2.4 Huấn luyện và NMS . . . 4
  - 2.4.1 Hàm mất mát đa nhiệm . . . 4
  - 2.4.2 Suy luận, candidate filtering và polygon NMS . . . 5
- 2.5 Baseline Otsu và Watershed . . . 5
  - 2.5.1 Otsu và connected components . . . 5
  - 2.5.2 Marker-controlled Watershed . . . 5
  - 2.5.3 So sánh khái niệm . . . 6

3 Fiji/ImageJ làm nền tảng tham chiếu 7
- 3.1 Plugin StarDist . . . 7
  - 3.1.1 Vai trò của ImageJ, Fiji và plugin . . . 7
  - 3.1.2 Thông số có ý nghĩa thực nghiệm . . . 7

3.2 Protocol tái lập và lần chạy thực tế 8
3.2.1 Các bước thao tác và bằng chứng cần lưu 9
3.3 Kiểm chứng Fiji–Python 10

4 Cài đặt Python **11**
4.1 Kiến trúc phần mềm 11
4.2 Phần nhóm tự cài đặt và phần tái sử dụng 11
4.3 Tái lập môi trường 11

5 Thiết kế thí nghiệm và metric **13**
5.1 Dữ liệu và split 13
5.2 Matching và metric 13
5.3 Protocol benchmark 14

6 Kết quả và thảo luận **15**
6.1 Benchmark tái lập được trên DSB2018 15
6.2 Cách đọc kết quả 18
6.3 Đối chiếu trực tiếp Fiji–Python: trả lời RQ2 18
6.4 Ảnh hưởng probability/NMS: trả lời RQ3 20
6.5 Đe doạ tới tính hợp lệ 21

7 Kết luận và hướng phát triển **22**

A Phân công công việc **24**

B Hướng dẫn tái lập đầy đủ **25**
B.1 Cài môi trường và kiểm thử 25
B.2 Tạo lại kết quả và artefact 25

C Mã nguồn cốt lõi **26**
C.1 Matching và metric instance 26
C.2 Star-convex geometry reference 31
C.3 Script thí nghiệm chạy thật 37
C.4 Runner StarDist ImageJ/Fiji 52

Danh sách hình vẽ

3.1 Minh chứng từng bước của workflow Fiji/ImageJ trên ca median. Từ trái sang phải: input TIFF, ảnh hiển thị percentile 1/99.8 (minh hoạ), Label Image TIFF StarDist, overlay prediction/ground truth và parameter/runtime thực tế. Đây là artefact đầu ra của plugin/evaluator, không phải mask minh hoạ tự vẽ. 9

6.1 Mean AP@0.50 và PQ@0.50 trên 50 ảnh test DSB2018. Error bar được trình bày trong Bảng 6.1. 16

6.2 Ca median được giữ cố định từ lần chạy gốc có clipping, không chọn lại theo kết quả mới. Màu biểu diễn ID instance, không phải lớp semantic. Otsu gộp nhiều nhân tiếp xúc; Watershed tách được một phần; StarDist bám theo instance ground truth sát hơn. 16

6.3 Ca low được giữ cố định từ lần chạy gốc, không khẳng định vẫn có AP thấp nhất sau cập nhật. Hình này được báo cáo cùng case trung vị để tránh chỉ chọn ví dụ thuận lợi. 17

6.4 Ba output Label Image thực tế của plugin StarDist Fiji/ImageJ: ca low, median và high đã chọn trước, đặt cạnh input và ground truth. Mỗi màu là một ID instance; figure được sinh trực tiếp từ TIFF plugin xuất. 17

6.5 Sai khác trực tiếp giữa hai đầu ra. ID nhãn được ghép một-một trước khi đo pixel agreement trên hợp foreground, tránh kết luận sai do đổi thứ tự ID. 19

6.6 Probability sweep trên 30 ảnh validation; error bar là SD giữa ảnh. 20

6.7 NMS sweep trên cùng validation, giữ probability mặc định. 20

6.8 Ví dụ ở ID validation đầu tiên đã chốt, không chọn theo score. Hai đầu mút probability/NMS kèm FP/FN minh hoạ candidate được giữ/bỏ khi đổi ngưỡng. Màu là ID instance; kết luận định lượng vẫn dựa toàn bộ 30 ảnh. 21

Danh sách bảng

6.1 Metric instance trên cùng 50 test images. Fiji được đánh giá riêng ở bảng đối chiếu. 15
6.2 Metric foreground, sai số đếm và runtime Python/baseline, mean ± SD trên 50 ảnh. 16

Chương 1

Giới thiệu

1.1 Bối cảnh và động cơ

Ảnh hiển vi thường chứa nhiều nhân tế bào sát nhau. Mask foreground nhị phân chỉ trả lời pixel nào thuộc tế bào, nhưng không trả lời pixel đó thuộc tế bào nào. Vì vậy, một vùng chạm nhau có thể bị gộp thành một object, làm sai số đếm, diện tích và các phép đo hình thái. Instance segmentation yêu cầu một nhãn riêng cho từng object, khác semantic segmentation và object detection chỉ dùng bounding box.

StarDist biểu diễn mỗi object bằng một đa giác star-convex, phù hợp với nhiều nhân tế bào dạng blob. So với cách phân đoạn bottom-up, các proposal hình học và NMS giúp tách object ngay cả khi ranh giới pixel mờ [6]. Tuy nhiên giả thiết star-convex không phù hợp hoàn toàn với object hình vòng, phân nhánh hoặc lõm sâu; đánh giá trên dữ liệu mục tiêu là bắt buộc.

1.2 Mục tiêu và đóng góp

Mục tiêu là nghiên cứu, cài đặt, thực nghiệm và đánh giá StarDist 2D với Fiji làm tham chiếu. Sản phẩm gồm: (i) mô hình toán học của StarDist, CNN/U-Net, Otsu và Watershed; (ii) protocol Fiji có log tham số và macro batch; (iii) package Python với CLI, baseline, geometry reference và metric; (iv) thiết kế thí nghiệm định lượng; và (v) báo cáo, presentation.

1.3 Câu hỏi nghiên cứu

RQ1 kiểm tra StarDist có tách object chạm nhau tốt hơn Otsu/Watershed không. RQ2 kiểm tra tính tương thích Fiji–Python khi giữ nguyên input và tham số. RQ3 đo độ nhạy của probability threshold và NMS threshold. Mỗi câu hỏi dùng test set cố định; validation chỉ dùng để chọn tham số.

Chương 2

Cơ sở lý thuyết

2.1 Bài toán instance segmentation

Cho ảnh $I : \Omega \to \mathbb{R}^C$, với miền pixel $\Omega = \{0, \dots, H-1\} \times \{0, \dots, W-1\}$ và C kênh. Ground truth là ảnh nhãn nguyên $Y : \Omega \to \{0, 1, \dots, M\}$: $Y(q) = 0$ là nền; mọi pixel mang cùng nhãn $m > 0$ tạo thành instance G_m. Mục tiêu là ước lượng $\hat{Y} = f_\theta(I)$ sao cho từng object có ID riêng, kể cả khi hai object cùng lớp chạm biên. Đây là khác biệt cốt lõi giữa ba nhiệm vụ thường bị nhầm lẫn:

- **Semantic segmentation** dự đoán foreground/background hoặc lớp tại mỗi pixel; hai nhân chạm nhau vẫn có thể là một vùng liên thông.
- **Object detection** trả bounding box và score, hữu ích để định vị nhưng không đủ cho diện tích, chu vi hay hình dạng nhân.
- **Instance segmentation** trả tập mask/polygon $\{P_j\}_{j=1}^{N}$; các P_j được phân biệt theo ID và là đầu vào đúng cho phép đếm.

Trong ảnh hiển vi, sai khác giữa foreground mask và instance label có ảnh hưởng trực tiếp đến định lượng: hai nhân dính nhau làm số object giảm một, diện tích tăng giả tạo và các đặc trưng hình thái bị trộn. Vì vậy, một Dice foreground cao chưa bảo đảm chất lượng instance; báo cáo cần matching theo từng object ở nhiều ngưỡng IoU.

2.1.1 Tiền xử lý và quy ước số học

Các model pretrained nhạy với thang cường độ. Với percentile thấp/cao a, b, chuẩn hoá dùng trong thí nghiệm là

$$I_{\text{norm}}(q) = \frac{I(q) - Q_a(I)}{Q_b(I) - Q_a(I) + \varepsilon}, \tag{2.1}$$

trong đó Q_a, Q_b là percentile và ε tránh chia cho 0. Nó không thay đổi toạ độ hay pixel size. Python dùng percentile tuyến tính NumPy, không cắt giá trị ngoài $[0, 1]$. Fiji cũng dùng $a = 1$, $b = 99.8$ và không cắt ngưỡng trên, nhưng CSBDeep Java dùng histogram percentile và chặn ngưỡng dưới ở 0. Vì vậy hai tiền xử lý gần nhau, không được coi là giống hệt nhau. Label output

luôn là TIFF 2D số nguyên, không phải ảnh RGB màu; màu chỉ dùng để hiển thị ID.

2.2 CNN và U-Net

2.2.1 Tích chập, receptive field và phi tuyến

CNN học các kernel thay vì thiết kế thủ công feature. Với feature map đầu vào $a^{(l-1)}$ và kernel $w^{(l)}$, giá trị ở lớp l là

$$a_{i,j,c}^{(l)} = \phi\left(b_c^{(l)} + \sum_{u,v,c'} w_{u,v,c',c}^{(l)} a_{i+u,j+v,c'}^{(l-1)}\right). \tag{2.2}$$

Hàm phi tuyến ϕ (ví dụ ReLU) cho phép xấp xỉ quan hệ phi tuyến giữa cấu trúc ảnh và nhãn. Khi nhiều lớp và pooling/strided convolution nối tiếp, một pixel đầu ra quan sát được vùng lân cận lớn hơn, gọi là *receptive field*. Ngữ cảnh này hữu ích để phân biệt đốm nhiễu với nhân tế bào, nhưng downsampling quá mạnh sẽ làm mất chi tiết biên.

2.2.2 U-Net làm backbone đa đầu ra

U-Net gồm encoder giảm dần độ phân giải để học ngữ cảnh, bottleneck, rồi decoder upsampling để khôi phục lưới pixel. Skip connection nối feature map cùng scale từ encoder sang decoder, giúp decoder dùng lại vị trí/biên mảnh thay vì chỉ dựa vào feature đã nén [5]. StarDist dùng U-Net như backbone chia sẻ và gắn hai head 1×1 convolution tại độ phân giải ảnh:

$$(\hat{p}(q), \hat{\mathbf{r}}(q)) = h_\theta(I)(q), \qquad \hat{p}(q) \in [0,1], \quad \hat{\mathbf{r}}(q) \in \mathbb{R}_+^K. \tag{2.3}$$

Head probability trả một kênh score; head hình học trả K khoảng cách tia. Vì hai task dùng cùng feature, score có thể đánh giá độ tin cậy proposal còn distance head mô tả biên object. Mạng *không* sinh trực tiếp một label ID cho mỗi pixel, vì ID không có thứ tự cố định giữa các ảnh.

2.3 Probability map và star-convex polygon

2.3.1 Giả thiết star-convex và target radial distance

Một tập S là star-convex đối với điểm c nếu mọi đoạn thẳng từ c đến điểm $x \in S$ đều nằm trong S, tức $[c, x] \subseteq S$. Đối với một pixel foreground q và K hướng đều nhau $\theta_k = 2\pi k/K$, target radial là

$$r_k(q) = \max\{t \geq 0 : q + t(\cos\theta_k, \sin\theta_k) \in G(q)\}, \quad k = 0, \ldots, K-1, \tag{2.4}$$

trong đó $G(q)$ là instance chứa q. Nói cách khác, ta đi từ q theo từng tia đến giao điểm đầu tiên với biên. Bài báo gốc dùng $K = 32$ [6]; K lớn hơn mô tả biên cong tốt hơn nhưng tăng số kênh,

bộ nhớ và số polygon phải xử lý trong NMS.

Khi mạng dự đoán $\hat{\mathbf{r}}(q)$, các đỉnh polygon proposal là

$$v_k(q) = q + \hat{r}_k(q)(\cos\theta_k, \sin\theta_k). \tag{2.5}$$

Nối $v_0, \ldots, v_{K-1}$ theo thứ tự góc tạo polygon xấp xỉ instance. Một object có thể sinh proposal từ nhiều pixel bên trong nó, nhưng proposal gần tâm thường ổn định hơn. Target probability $p(q)$ là Euclidean distance transform (EDT) chuẩn hoá riêng trong từng instance: gần biên có giá trị xấp xỉ 0, trung tâm có giá trị cao. Do đó probability map vừa là foreground confidence vừa thiên về những tâm sinh polygon tốt.

2.3.2 Khả năng biểu diễn và giới hạn

Star-convex polygon phù hợp nhân tế bào/bào quan dạng khối tròn hoặc bầu dục, và tách hai nhân sát nhau bằng hai polygon chồng cạnh. Ngược lại, hình vòng, nhánh cây hoặc lõm sâu có thể không nhìn thấy toàn bộ biên từ một tâm; ray đi qua nền rồi vào lại object làm biểu diễn không đúng. Lỗi khi đó là hạn chế mô hình hình học, không chỉ là lỗi threshold. Cần kiểm tra overlay và cân nhắc Watershed, Mask R-CNN hay fine-tuning đối với dữ liệu có hình dạng này.

2.4 Huấn luyện và NMS

2.4.1 Hàm mất mát đa nhiệm

Mạng học probability $\hat{p}$ và vector distance $\hat{\mathbf{r}}$. Với target $p, \mathbf{r}$, một dạng objective điển hình là

$$\mathcal{L}_{\text{prob}} = -\frac{1}{|\Omega|}\sum_{q\in\Omega}[p(q)\log\hat{p}(q) + (1-p(q))\log(1-\hat{p}(q))], \tag{2.6}$$

$$\mathcal{L}_{\text{dist}} = \frac{\sum_q p(q)\,\|\mathbf{r}(q) - \hat{\mathbf{r}}(q)\|_1}{\sum_q p(q) + \varepsilon}, \tag{2.7}$$

$$\mathcal{L} = \lambda_p\mathcal{L}_{\text{prob}} + \lambda_r\mathcal{L}_{\text{dist}}. \tag{2.8}$$

Trọng số $p(q)$ giảm tác động của pixel biên, nơi tia ngắn và nhạy với annotation, đồng thời nhấn vào các tâm. Khi dùng pretrained model, các trọng số đã được học trước; thí nghiệm này chỉ làm inference, không tuyên bố train lại CNN.

2.4.2 Suy luận, candidate filtering và polygon NMS

Tại inference, tạo candidate $C = \{q : \hat{p}(q) \geq t_p\}$. Mỗi $q \in C$ được decode theo (2.5); sau đó sắp giảm dần theo $\hat{p}(q)$. Với proposal được giữ A và proposal xét B, overlap là

$$\text{overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}. \tag{2.9}$$

NMS giữ proposal score cao nhất còn lại và loại mọi proposal có $\text{overlap}(A, B) > t_{\text{nms}}$; lặp đến khi hết candidate. Độ đo này của StarDist 2D khác IoU dùng để đánh giá, vốn chuẩn hoá bằng hợp. Các polygon được giữ được raster hoá thành label ID không chồng nhau. Độ phức tạp CNN gần tuyến tính theo số pixel, còn hậu xử lý phụ thuộc số candidate và cặp polygon; vì vậy t_p, tile size và NMS ảnh hưởng runtime.

Hai threshold có ý nghĩa khác nhau, cần ghi tách biệt trong log. t_p thấp quá tạo nhiều positive giả và tăng chi phí NMS; cao quá làm mất object mờ. Ngưỡng t_{nms} thấp triệt proposal mạnh (dễ bỏ sót object sát nhau); cao dễ giữ trùng lặp. Các giá trị thực tế ở đây là $t_p = 0.479071$ và $t_{\text{nms}} = 0.3$, được lấy từ cấu hình model, không tune trên test.

2.5 Baseline Otsu và Watershed

2.5.1 Otsu và connected components

Otsu giả định histogram có thể tách thành hai lớp bằng ngưỡng t. Gọi ω_0, ω_1 là xác suất hai lớp và μ_0, μ_1 là trung bình cường độ của chúng, Otsu chọn ngưỡng toàn cục bằng cách cực đại hoá phương sai giữa lớp [4]:

$$t^* = \arg\max_t \ \sigma_B^2(t), \qquad \sigma_B^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2. \tag{2.10}$$

Sau threshold, lọc vùng nhỏ và connected components 8-neighbour tạo label. Điểm mạnh là không cần model, nhanh và có thể giải thích; điểm yếu là nền không đồng nhất/ảnh ít tương phản làm histogram không còn bimodal. Quan trọng hơn, hai object chạm nhau thành một component nên Otsu thường có Dice foreground tốt nhưng AP instance và count kém.

2.5.2 Marker-controlled Watershed

Watershed bổ sung bước tách object trên foreground mask F. Đầu tiên tính $D = \text{EDT}(F)$; đỉnh địa phương của D là marker gần tâm object. Thuật toán "ngập" landscape $-D$ từ các marker và đặt watershed line nơi hai vùng gặp nhau [8]. Pipeline triển khai là: (1) Otsu tạo F; (2) morphology/loại vùng nhỏ; (3) peak-local-max với khoảng cách tối thiểu d; (4) watershed của $-D$ bị ràng buộc trong F; (5) relabel output. Nó tách được nhân tiếp xúc tốt hơn Otsu nhưng

over-segment khi marker nhiễu, under-segment khi hai đỉnh quá gần hoặc một nhân không có cực đại rõ. Vì vậy d phải chọn trên validation thay vì test.

2.5.3 So sánh khái niệm

Otsu và Watershed là phương pháp bottom-up dựa trên cường độ/địa hình; StarDist là phương pháp learned proposal có prior star-convex. Baseline không cạnh tranh công bằng về lượng thông tin học được, nhưng cần thiết để xác định mức cải thiện thực sự so với xử lý ảnh cổ điển. Các output đều được chuẩn hoá thành label TIFF trước khi dùng chung metric.

Chương 3

Fiji/ImageJ làm nền tảng tham chiếu

3.1 Plugin StarDist

Plugin StarDist của Fiji cung cấp inference cho model 2D pretrained/custom, percentile normalization, probability threshold, overlap/NMS threshold, tile và output label image/ROI [2]. Fiji là công cụ tham chiếu cho thao tác trực quan và đo đạc, không thay thế đánh giá định lượng.

3.1.1 Vai trò của ImageJ, Fiji và plugin

ImageJ là nền tảng xử lý ảnh có ImagePlus, ROI Manager, macro recorder và hệ sinh thái plugin. Fiji là bản phân phối ImageJ đã tích hợp nhiều plugin khoa học và cơ chế update site; StarDist là plugin inference chạy trong runtime đó. Vì vậy, "mở bằng Fiji" không tự động bảo đảm tái lập: phải ghi phiên bản plugin, model, Java/TensorFlow, input, chuẩn hoá, threshold, tile và kiểu output. Trong báo cáo này, output chuẩn để đối chiếu là **Label Image TIFF**: nền có ID 0 và mỗi instance có ID nguyên dương. ROI Manager chỉ phục vụ quan sát, không dùng thay label để tính metric.

3.1.2 Thông số có ý nghĩa thực nghiệm

Tham số	Giá trị lần chạy tham chiếu	Ý nghĩa/điểm cần kiểm tra
Model	Versatile (fluorescent nuclei)	Phải phù hợp modality; không dùng model H&E cho ảnh huỳnh quang.
Normalize	percentile 1 / 99.8	Cùng percentile đặt trước, nhưng khác thuật toán percentile và xử lý ngưỡng dưới.

Tham số	Giá trị lần chạy tham chiếu	Ý nghĩa/điểm cần kiểm tra
Probability threshold	0.479071	Lọc candidate trước NMS; không tối ưu lại trên ba ảnh kiểm chứng.
NMS overlap	0.3	Khử proposal polygon trùng; khác với IoU threshold khi đánh giá.
Tiles / exclude boundary	1 / 2 px	Tile ảnh hưởng bộ nhớ và seam; boundary exclusion thay đổi object nằm sát biên.
Output	Label Image TIFF	Phải cùng kích thước với input/ground truth; không lưu PNG RGB để đo.

3.2 Protocol tái lập và lần chạy thực tế

Ngoài protocol GUI, nhóm chạy command StarDist đã công bố `de.csbdresden:StarDist_:0.3.0-scijava` trong ImageJ 2.9.0 headless, với ImageJ 1.53t legacy, Eclipse Temurin 8.0.504 và TensorFlow 1.12 CPU. Đây là chính artifact plugin dùng cho Fiji/ImageJ; Java 8 cần thiết vì TensorFlow ImageJ 1.12 là runtime cũ. Ba ảnh low/median/high được chọn trước từ test split theo AP của StarDist Python, không tune threshold trên ba ảnh này. Model là *Versatile (fluorescent nuclei)*, percentile $1/99.8$, probability 0.479071, overlap/NMS 0.3, một tile và boundary exclusion 2. Plugin xuất Label Image TIFF cùng kích thước input. Mỗi lần chạy và runtime được ghi trong `fiji/run_log_dsb2018_reference.csv`; runner Java là `fiji/scripts/FijiStarDistRunner.java`.

File `fiji/workflows/stardist_fiji_protocol.md` vẫn là hướng dẫn thao tác Fiji GUI; macro `fiji/macros/stardist_batch.ijm` yêu cầu dùng Fiji Recorder để chèn đúng lệnh của phiên bản plugin, tránh đoán option macro.

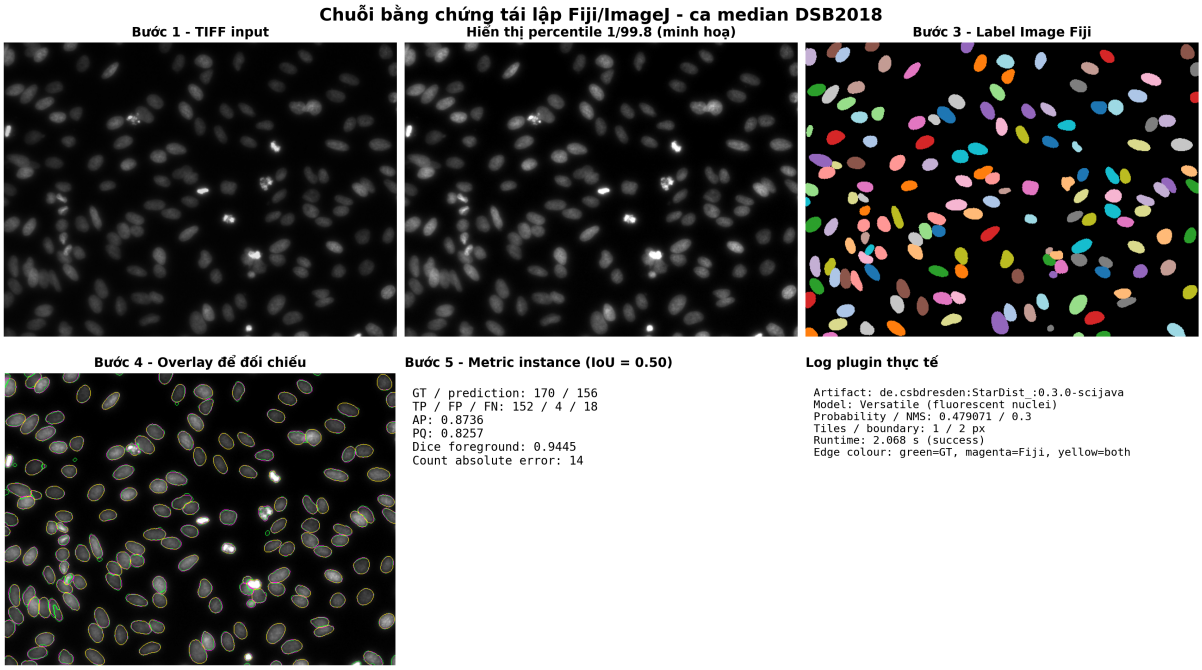
Đợt tái tạo ngày 24/10/2026 bổ sung `fiji/pom.xml` và `scripts/run_fiji_reference.py`: Maven 3.9.9 dựng classpath trên máy macOS Intel với Java 8u504, runner xuất lại ba TIFF và log mới. Các file native JNI và framework TensorFlow 1.12 được trích từ JAR vào cùng thư mục để nạp đủ thư viện trên macOS. `results/fiji_rerun/reproduction_check.csv` xác nhận ba mảng label trùng hoàn toàn từng pixel với nhãn tham chiếu lịch sử. Manifest JAR/SHA256 và metric chạy lại lưu trong `results/fiji_rerun/`. Runtime mới giữ riêng, không thay số runtime lịch sử trên phần cứng khác.

3.2.1 Các bước thao tác và bằng chứng cần lưu

Quy trình GUI được áp dụng theo đúng thứ tự sau. Mỗi bước có artefact tương ứng trong repository để người chấm kiểm tra thay vì chỉ tin vào mô tả.

1. Mở TIFF 2D và xác nhận kích thước/bit-depth/channel bằng `Image > Show Info`. Lưu tên file input và không resize ảnh.
2. Trong menu `Plugins > StarDist`, chọn `StarDist 2D`. Chọn model, nhập percentile, probability, NMS, tile và bật `Label Image`.
3. Chạy inference; sau khi output xuất hiện, kiểm tra input và label cùng chiều cao/rộng, quan sát overlay/ROI để phát hiện merge hoặc split bất thường.
4. Lưu label TIFF nguyên gốc; không export screenshot hay PNG để đánh giá. Ghi runtime và tất cả parameter vào run log.
5. Chạy evaluator Python trên chính TIFF đó. Kiểm tra TP/FP/FN, overlay và metric trước khi tổng hợp mean $\pm$ SD.

Hình 3.1 là bằng chứng trực quan của năm đầu ra từ workflow thật ở ca median: TIFF input, input tăng tương phản để minh hoạ, label image, overlay biên đối chiếu ground truth và block tham số/runtime trích từ run log. Panel được dựng từ file đầu vào/đầu ra thực tế. Ảnh tăng tương phản chỉ phục vụ hiển thị, không phải tensor chuẩn hoá lấy trực tiếp từ plugin. Các output ba ca low/median/high đầy đủ tiếp tục được trình bày ở Hình 6.4.



Hình 3.1: Minh chứng từng bước của workflow Fiji/ImageJ trên ca median. Từ trái sang phải: input TIFF, ảnh hiển thị percentile 1/99.8 (minh hoạ), Label Image TIFF StarDist, overlay prediction/ground truth và parameter/runtime thực tế. Đây là artefact đầu ra của plugin/evaluator, không phải mask minh hoạ tự vẽ.

3.3 Kiểm chứng Fiji–Python

Đánh giá Fiji với ground truth đo chất lượng phân đoạn; so sánh Fiji với Python đo độ tương ứng hai triển khai. Hai phép đo được lưu riêng trong `results/fiji_reference/` và `results/cross_platform/`. Không coi điểm AP tương tự với ground truth là bằng chứng hai output giống nhau. Kết quả đối chiếu trực tiếp và khác biệt tiền xử lý được thảo luận ở Chương 6.

Chương 4

Cài đặt Python

4.1 Kiến trúc phần mềm

Package `src/xla_gr03` tách tiền xử lý, I/O TIFF, Otsu/Watershed, geometry, StarDist wrapper, metrics và CLI. Mọi output là label integer 2D. Lệnh ‘xla-gr03 segment‘ chạy một phương pháp; ‘xla-gr03 evaluate‘ xuất JSON metric. Unit tests bao phủ geometry, metric, preprocessing, I/O và baseline.

4.2 Phần nhóm tự cài đặt và phần tái sử dụng

Nhóm tự cài đặt EDT chuẩn hoá theo instance, ray distance, polygon decode, polygon NMS raster hoá, IoU matrix và hàm đánh giá instance dùng bộ giải Hungarian của SciPy. Các hàm geometry là bản tham chiếu giáo dục, chưa thay hậu xử lý chính thức trong inference. CNN/model pretrained được gọi từ thư viện StarDist chính thức [7]. Sự phân biệt này tránh tuyên bố sai rằng nhóm tự train lại toàn bộ mạng.

Listing 4.1: Ví dụ chạy một baseline

```
xla-gr03 segment data/raw/image_001.tif results/otsu_labels.tif \
  --method otsu --foreground bright --min-size 20
xla-gr03 evaluate data/ground_truth/image_001.tif results/otsu_labels
   .tif \
  --thresholds 0.5 0.75 0.9
```

4.3 Tái lập môi trường

Môi trường đề xuất dùng Python 3.11, NumPy, SciPy, scikit-image, tifffile, pytest và ruff; extra ‘stardist‘ thêm TensorFlow, csbdeep và StarDist. Chạy ‘conda env create -f environment.yml‘, sau đó ‘pytest‘ và ‘ruff check .‘. Model weight và dữ liệu nội bộ/raw không được commit; ngoại

lệ là bộ benchmark DSB2018 đã giải nén (497 cặp TIFF, xấp xỉ 194 MiB) được versioned tại `data/dsb2018/dsb2018/` để tái lập đúng kết quả báo cáo. Tham số mặc định được ghi trong CLI, constants của script và provenance hiệu lực của mỗi lần chạy. File YAML không được dùng đã được bỏ để tránh hai nguồn tham số.

Chương 5

Thiết kế thí nghiệm và metric

5.1 Dữ liệu và split

Dataset phải có instance ground truth hợp pháp, ví dụ DSB2018 hoặc MoNuSeg. Raw image và label TIFF cùng basename nằm trong ‘data/raw‘ và `data/ground_truth`. Development dùng để debug; validation chọn tham số; test chỉ đánh giá cuối. Không tối ưu threshold trên test set.

5.2 Matching và metric

Gọi G_i là instance ground truth thứ i và P_j là instance prediction thứ j. Trước hết code bỏ nhãn nền 0, lập ma trận giao nhau pixel-wise rồi tính

$$J_{ij} = \mathrm{IoU}(G_i, P_j) = \frac{|G_i \cap P_j|}{|G_i \cup P_j|}. \tag{5.1}$$

Không được ghép một prediction với nhiều ground truth hoặc ngược lại, vì điều đó có thể che giấu merge/split. Hungarian assignment tìm tập cặp một-một có tổng IoU lớn; chỉ những cặp có $J_{ij} \geq \tau$ mới là TP. Ground truth chưa ghép là FN, prediction chưa ghép là FP. Cách làm này phản ánh đúng lỗi tách instance: một prediction gộp hai nhân chỉ có thể khớp tối đa một nhân và nhân còn lại là FN.

Tại mỗi $\tau \in \{0.50, 0.75, 0.90\}$, các chỉ số là

$$P = TP/(TP + FP), \qquad R = TP/(TP + FN), \tag{5.2}$$

$$F1 = 2TP/(2TP + FP + FN), \qquad AP_\tau = TP/(TP + FP + FN). \tag{5.3}$$

Ở đây AP_τ là tên metric theo benchmark StarDist/Data Science Bowl, không phải area under precision–recall curve. $\tau = 0.50$ trả lời “có nhận đúng object không”, còn $\tau = 0.90$ đòi biên khớp rất chính xác. Bởi vậy, chỉ báo cáo một ngưỡng có thể che mất lỗi hình dạng.

Panoptic Quality kết hợp nhận diện và chất lượng mask [3]:

$$PQ_\tau = \frac{\sum_{(i,j)\in \mathrm{TP}} J_{ij}}{TP + \frac{1}{2}FP + \frac{1}{2}FN}. \tag{5.4}$$

Tử số là segmentation quality trên các cặp khớp; mẫu số phạt lỗi detection. Ngoài ra, foreground Dice $2|F_G \cap F_P|/(|F_G| + |F_P|)$ và foreground IoU đo vùng nhân hợp nhất, nhưng không phân biệt ID instance. Sai số đếm là $|N_P - N_G|$; runtime được đo per image và ghi rõ có/không có tải model. Toàn bộ metric tính từng ảnh trước, rồi báo cáo mean $\pm$ standard deviation trên đúng số ảnh của từng stratum.

5.3 Protocol benchmark

Otsu, Watershed và StarDist Python chạy trên toàn bộ 50 test images. Watershed được chọn bằng validation; StarDist dùng default đã đóng gói. StarDist Fiji/ImageJ chạy trên ba ca low/median/high cố định để kiểm chứng cross-platform với output label thật, không được gộp vào ước lượng 50 ảnh. Báo cáo mean $\pm$ SD, số ảnh và runtime riêng cho từng stratum. Script `scripts/run_dsb2018_benchmark.py` sinh benchmark 50 ảnh; script `scripts/evaluate_fiji_reference.py` đọc Label Image Fiji rồi đo metric.

Chương 6

Kết quả và thảo luận

6.1 Benchmark tái lập được trên DSB2018

Nhóm chạy ba phương pháp trên 50 ảnh của test split trong archive DSB2018 do StarDist công bố [1]. Ground truth là instance label TIFF; không dùng binary mask. Watershed chọn ‘min_distance‘ bằng sweep trên 30 ảnh lấy mẫu seed 42 từ 447 ảnh train, đạt AP@0.50 validation cao nhất tại giá trị 9. Danh sách 30 image ID được lưu trong validation_images.csv. Các 50 ảnh test không tham gia chọn tham số. StarDist Python dùng model `2D_versatile_fluo`, percentile $1/99.8$, default probability threshold 0.479071 và NMS threshold 0.3 từ model. Runtime CPU tính cả chuẩn hoá, inference và hậu xử lý sau warm-up; không tính đọc/ghi file và tải model. Các thiết bị TensorFlow, phiên bản package, hash weight và hash dữ liệu được lưu trong provenance và manifest. Lệnh benchmark Python sinh toàn bộ CSV, figure và provenance; đường dẫn tái lập được liệt kê trong README và phụ lục tái lập.

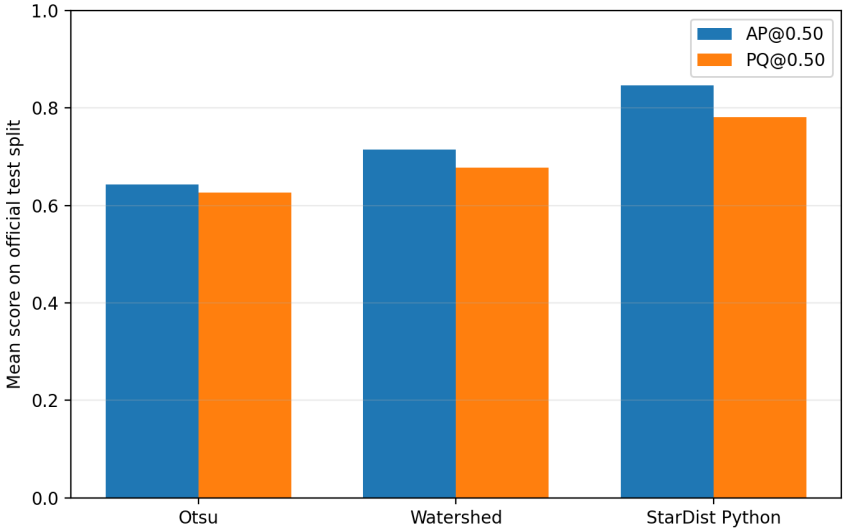
Bảng 6.1: Metric instance trên cùng 50 test images. Fiji được đánh giá riêng ở bảng đối chiếu.

Method	AP@.50	AP@.75	AP@.90	PQ@.50
Otsu	0.643 ± 0.238	0.453 ± 0.276	0.223 ± 0.221	0.627 ± 0.208
Watershed	0.715 ± 0.199	0.501 ± 0.266	0.214 ± 0.207	0.677 ± 0.166
StarDist Python	0.846 ± 0.132	0.692 ± 0.238	0.305 ± 0.313	0.782 ± 0.117

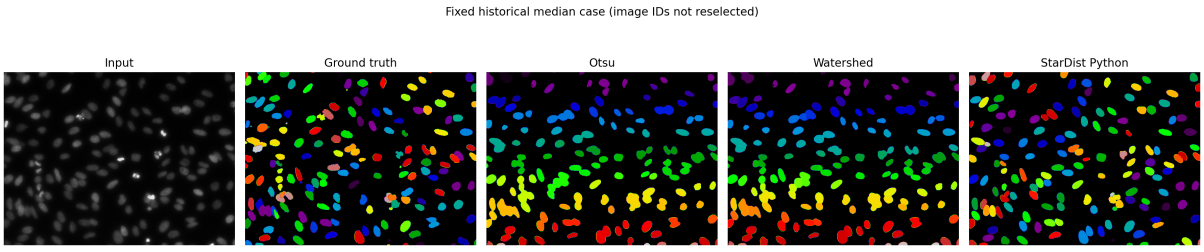
Bảng 6.2: Metric foreground, sai số đếm và runtime Python/baseline, mean ± SD trên 50 ảnh.

Method	Dice foreground	MAE count	Runtime (s/image)
Otsu	0.895 ± 0.070	14.06 ± 25.28	0.0327 ± 0.0232
Watershed	0.894 ± 0.071	6.94 ± 10.56	0.0647 ± 0.0462
StarDist Python	0.915 ± 0.050	3.48 ± 3.79	0.3277 ± 0.2098

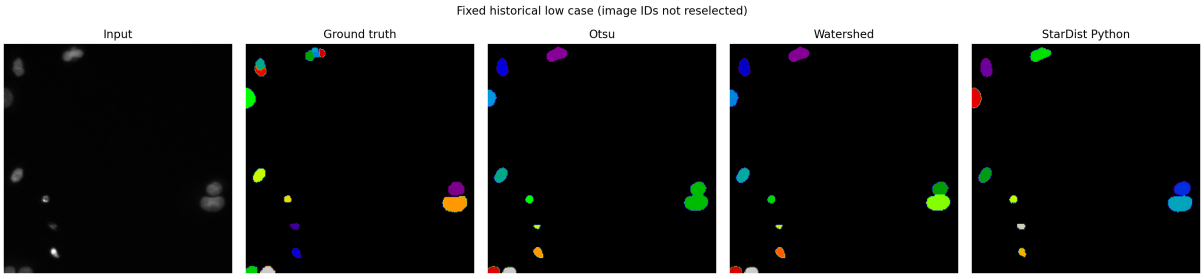
Ghi chú về công bằng so sánh. Dòng Fiji/ImageJ là kiểm chứng triển khai trên ba ca đã cố định; không phải ước lượng chung cho 50 test images nên không dùng để xếp hạng trực tiếp với ba dòng $n = 50$. Runtime Fiji đo command CPU, bao gồm inference/model initialization của process Java; runtime Python bao gồm tiền xử lý, inference và hậu xử lý sau warm-up. Không dùng hai khoảng đo này để xếp hạng tốc độ Fiji và Python.



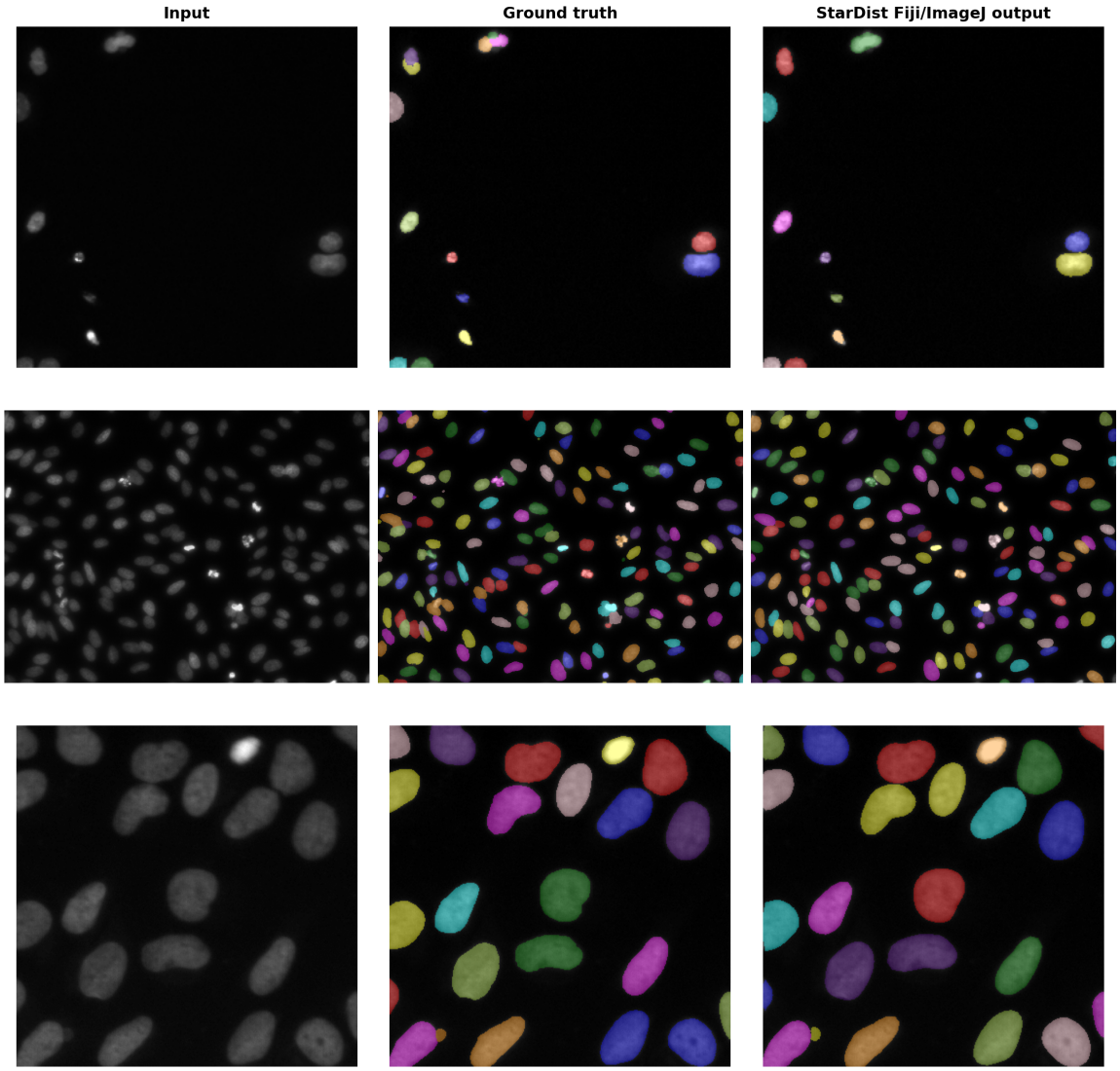
Hình 6.1: Mean AP@0.50 và PQ@0.50 trên 50 ảnh test DSB2018. Error bar được trình bày trong Bảng 6.1.



Hình 6.2: Ca median được giữ cố định từ lần chạy gốc có clipping, không chọn lại theo kết quả mới. Màu biểu diễn ID instance, không phải lớp semantic. Otsu gộp nhiều nhân tiếp xúc; Watershed tách được một phần; StarDist bám theo instance ground truth sát hơn.



Hình 6.3: Ca low được giữ cố định từ lần chạy gốc, không khẳng định vẫn có AP thấp nhất sau cập nhật. Hình này được báo cáo cùng case trung vị để tránh chỉ chọn ví dụ thuận lợi.



Hình 6.4: Ba output Label Image thực tế của plugin StarDist Fiji/ImageJ: ca low, median và high đã chọn trước, đặt cạnh input và ground truth. Mỗi màu là một ID instance; figure được sinh trực tiếp từ TIFF plugin xuất.

6.2 Cách đọc kết quả

Trên 50 ảnh test, StarDist đạt AP@0.50 0.846 ± 0.132 và PQ 0.782 ± 0.117, so với Watershed (0.715 ± 0.199; 0.677 ± 0.166) và Otsu (0.643 ± 0.238; 0.627 ± 0.208). Hai baseline vẫn phủ foreground khá tốt nhưng AP instance thấp hơn: kết quả này cho thấy Dice toàn foreground không đủ để đánh giá tách object. Watershed giảm MAE đếm từ 14.06 xuống 6.94 so với Otsu; StarDist đạt 3.48. Ở $\tau = 0.90$, metric của cả ba giảm mạnh, phản ánh yêu cầu chính xác biên rất nghiêm ngặt. StarDist có runtime CPU 0.3277 s/ảnh so với 0.0647 s/ảnh của Watershed, đổi lại tách instance tốt hơn trên benchmark này. Trên ba ca kiểm chứng, plugin Fiji/ImageJ đạt AP@0.50 0.801 ± 0.244, PQ 0.788 ± 0.160 và Dice 0.950 ± 0.017; các TIFF/metric chứng minh workflow chạy được; bảng đối chiếu trực tiếp bổ sung đo tương ứng hai đầu ra, không thay thế benchmark $n = 50$.

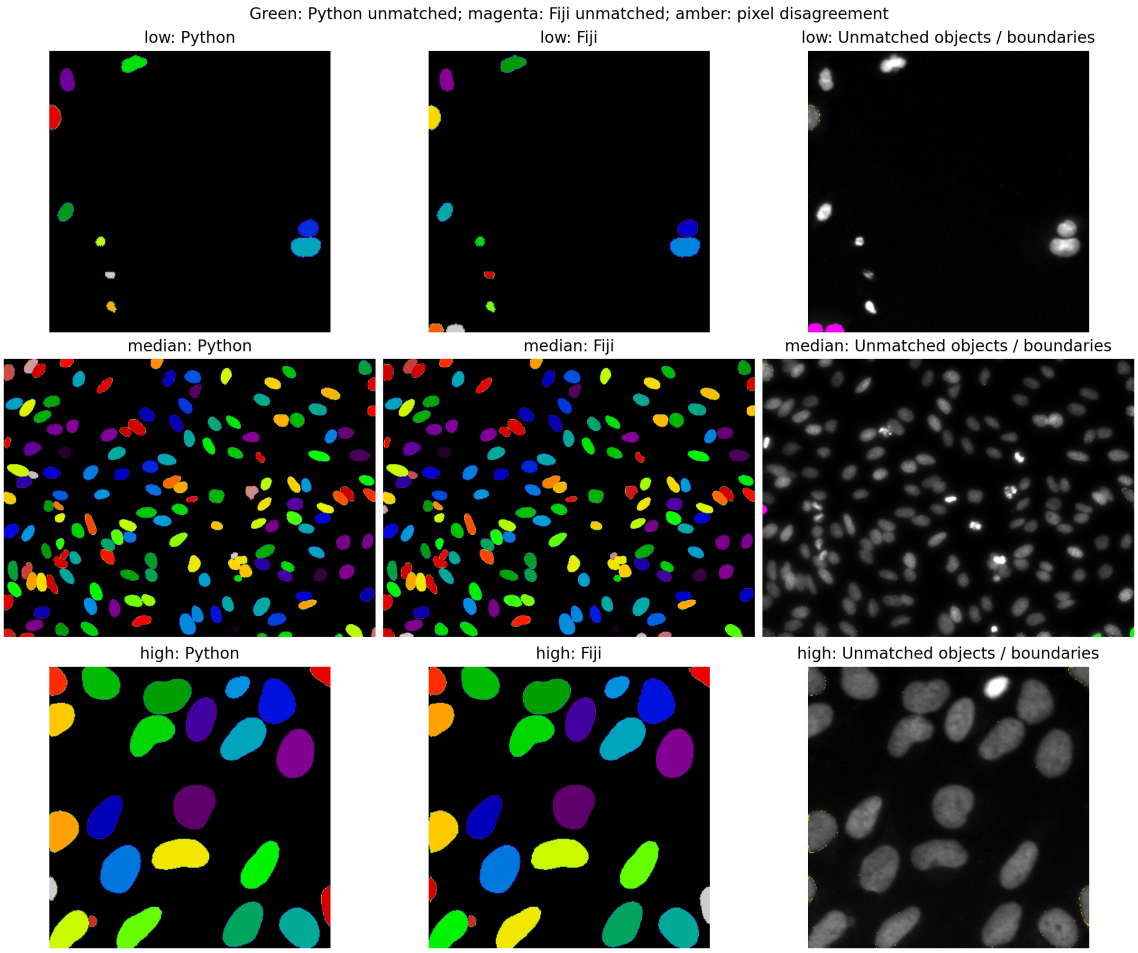
6.3 Đối chiếu trực tiếp Fiji–Python: trả lời RQ2

Script `scripts/compare_fiji_python.py` ghép trực tiếp label Python với label Fiji trên ba image ID cố định. Nhãn Python chỉ là tham chiếu so sánh, không thay ground truth sinh học. Tiêu chí mô tả dùng trong script là F1 tại IoU 0.50 ít nhất 0.95 và Dice foreground ít nhất 0.95. Đây là ngưỡng kỹ thuật cho ba ca, không phải kiểm định tương đương thống kê.

Ca	N_{Python}	N_{Fiji}	F1@.50	IoU ghép TB	Dice
low	9	11	0.9000	0.9956	0.9405
median	157	156	0.9904	0.9975	0.9970
high	24	24	1.0000	0.9903	0.9980

Ca median và high đạt tiêu chí mô tả; ca low không đạt: Python có 9 object, Fiji có 11 object, F1 0.900 và Dice 0.940. Vì vậy kết luận đúng là hai triển khai gần nhau ở hai ca, chưa đồng nhất trên cả ba. Ca low còn hai object Fiji không ghép được với Python, cần kiểm tra tín hiệu/biên của từng object thay vì chỉ nhìn trung bình. Overlay đánh dấu object Python chưa ghép bằng xanh lá, Fiji chưa ghép bằng tím, sai khác pixel/biên sau matching bằng vàng.

Python dùng percentile tuyến tính NumPy và phép affine không clipping. CSBDeep Java 0.6.0 dùng histogram percentile và chặn giá trị âm về 0 dù `clip=false`. Fiji còn đặt exclude boundary 2 pixel; hai nền tảng dùng TensorFlow khác phiên bản. Những khác biệt có thật này được lưu trong CSV và provenance; thí nghiệm hiện tại chưa cô lập từng nguyên nhân nên không quy hai object chênh lệch cho riêng một bước.

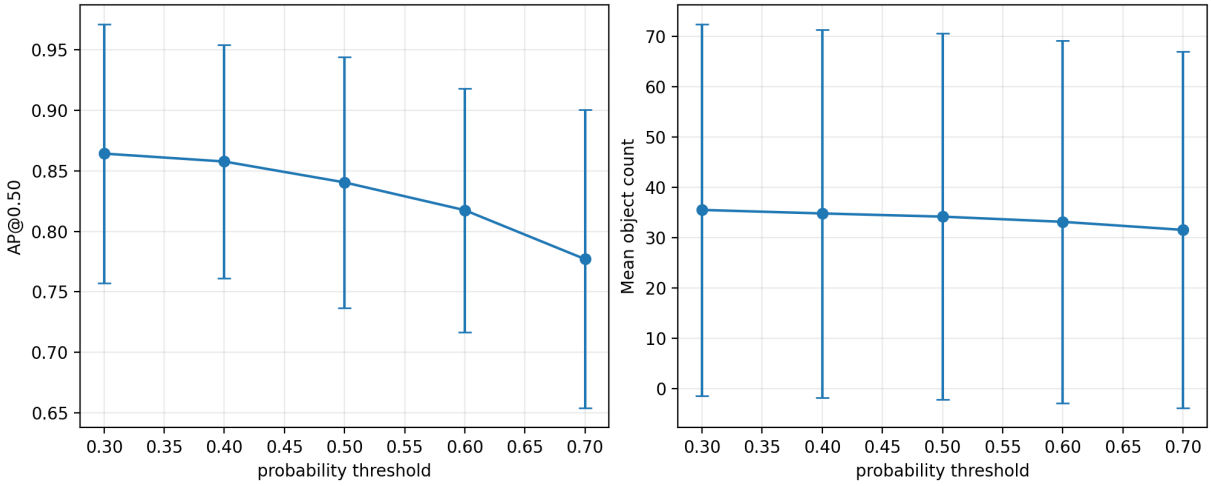


Hình 6.5: Sai khác trực tiếp giữa hai đầu ra. ID nhãn được ghép một-một trước khi đo pixel agreement trên hợp foreground, tránh kết luận sai do đổi thứ tự ID.

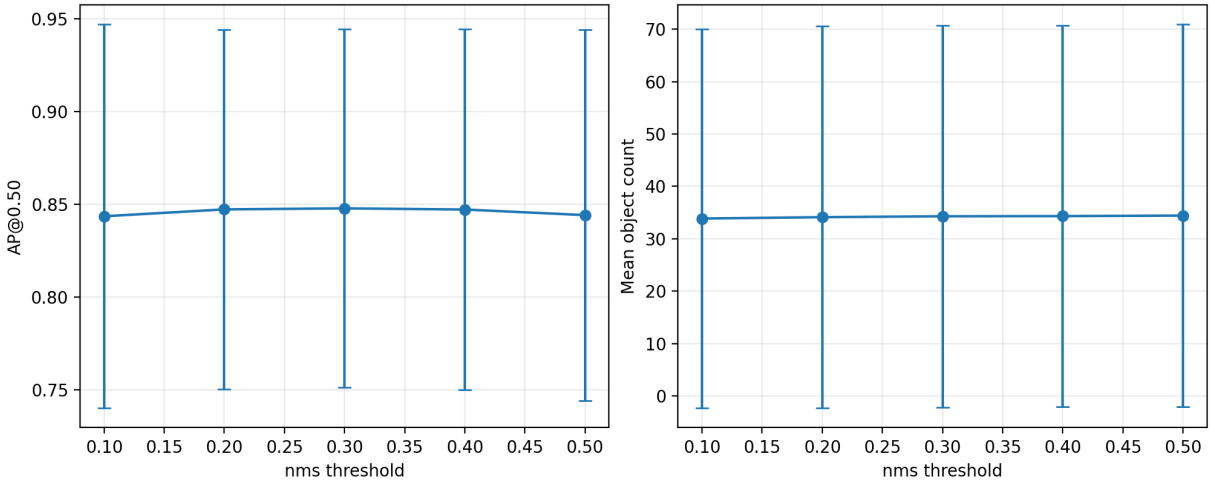
6.4 Ảnh hưởng probability/NMS: trả lời RQ3

Script `scripts/run_validation_ablation.py` đánh giá 5 probability threshold và 5 NMS threshold trên đúng 30 ảnh train-validation đã chốt. Mỗi ảnh chỉ tính CNN một lần rồi thay tham số hậu xử lý; do đó sweep không báo thời gian inference. CSV có mean và SD của AP, F1, PQ, số object, FP, FN và sai số đếm. Không sử dụng ảnh test để chọn ngưỡng.

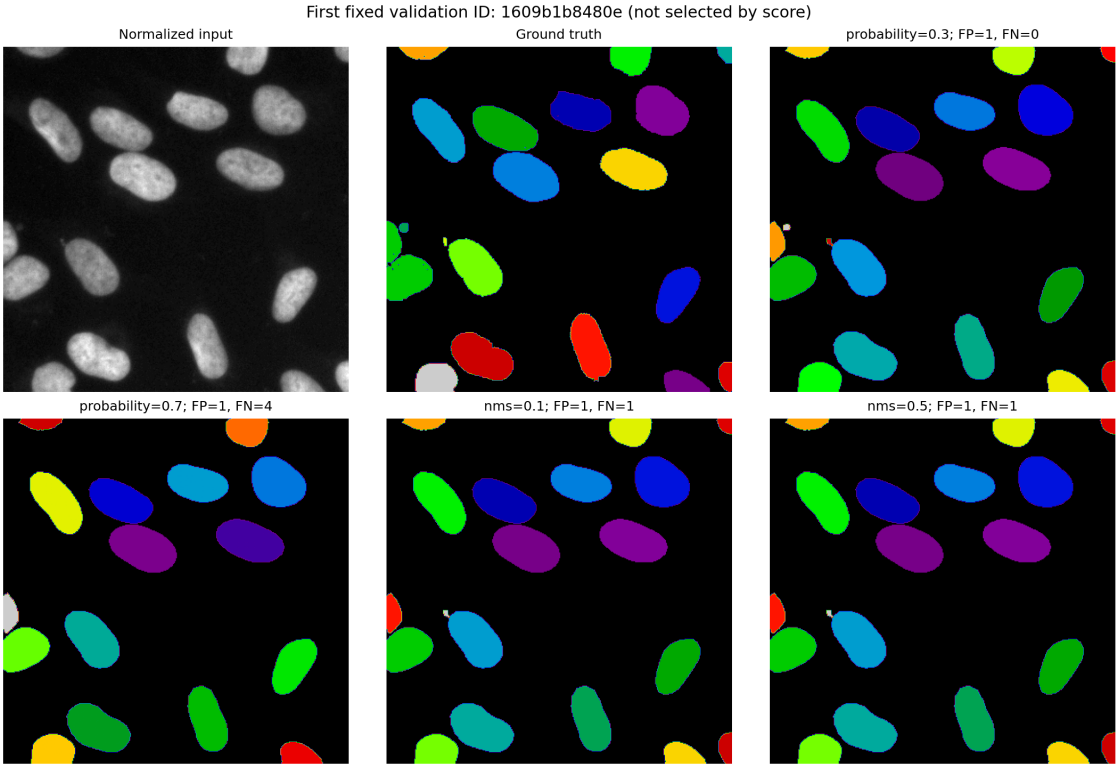
Khi tăng probability threshold từ 0.3 lên 0.7, AP@0.50 validation giảm từ 0.864 xuống 0.777; số object trung bình giảm từ 35.47 xuống 31.50; FN tăng từ 3.13 lên 6.47. Ngưỡng cao bỏ thêm candidate yếu và có thể bỏ sót nhân. Quét NMS giữ probability mặc định 0.479071; quét probability giữ NMS 0.3. Hai biểu đồ cho thấy cả thay đổi chất lượng và thay đổi số lượng, kèm SD giữa ảnh. Benchmark test vẫn giữ ngưỡng model công bố, không đổi theo sweep này.



Hình 6.6: Probability sweep trên 30 ảnh validation; error bar là SD giữa ảnh.



Hình 6.7: NMS sweep trên cùng validation, giữ probability mặc định.



Hình 6.8: Ví dụ ở ID validation đầu tiên đã chốt, không chọn theo score. Hai đầu mút probability/NMS kèm FP/FN minh hoạ candidate được giữ/bỏ khi đổi ngưỡng. Màu là ID instance; kết luận định lượng vẫn dựa toàn bộ 30 ảnh.

6.5 Đe doạ tới tính hợp lệ

Các kết luận được giới hạn bởi bốn yếu tố. (i) Model pretrained có thể không phù hợp stain hoặc optics của dữ liệu mới; chưa có fine-tuning trong repo; dữ liệu huấn luyện của weight pretrained có nguồn từ DSB2018, chưa xác minh độc lập danh sách image ID so với 50 ảnh test. Vì vậy kết quả là đánh giá pretrained trên split công bố, chưa phải bằng chứng tổng quát hoá độc lập khỏi dữ liệu huấn luyện weight. (ii) Kết quả Fiji chỉ có ba ca cố định, vì vậy là kiểm chứng tương thích chứ không phải ước lượng hiệu năng cùng cỡ mẫu $n = 50$ của Python. (iii) Khác biệt Fiji–Python có thể do version, normalization, tile hoặc NMS, không nhất thiết do thuật toán; do đó nhóm lưu output label, log và tham số của từng ca. (iv) Runtime giữa hai nền tảng không hoàn toàn đồng nhất vì Fiji bao gồm chi phí khởi tạo plugin và môi trường thực thi. Cuối cùng, instance có hình không star-convex là giới hạn cấu trúc; fine-tuning hoặc phương pháp khác cần được cân nhắc.

Chương 7

Kết luận và hướng phát triển

StarDist biến dự đoán pixel dày đặc thành tập polygon có định danh instance, phù hợp với ảnh nhân tế bào đông đúc. Repo cung cấp pipeline kiểm chứng được giữa Fiji/ImageJ và Python, baseline cổ điển, metric instance, label TIFF thực tế và run log. Hướng phát triển là fine-tuning bằng annotation nội bộ, ablation số tia, robustness với noise và đánh giá 3D.

Tài liệu tham khảo

[1] Juan C. Caicedo, Allen Goodman, Kyle W. Karhohs, et al. Nucleus segmentation across imaging experiments: the 2018 data science bowl. *Nature Methods*, 16(12):1247–1253, 2019. doi: 10.1038/s41592-019-0612-7.

[2] ImageJ. Stardist. https://imagej.net/plugins/stardist, 2026. Accessed: 2026-10-03.

[3] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. Panoptic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9404–9413, 2019. doi: 10.1109/CVPR.2019.00963.

[4] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979. doi: 10.1109/TSMC.1979.4310076.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.

[6] Uwe Schmidt, Martin Weigert, Coleman Broaddus, and Gene Myers. Cell detection with star-convex polygons. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018*, pages 265–273. Springer, 2018. doi: 10.1007/978-3-030-00934-2_30.

[7] StarDist contributors. Stardist – object detection with star-convex shapes. https://github.com/stardist/stardist, 2026. Accessed: 2026-10-03.

[8] Luc Vincent and Pierre Soille. Watersheds in digital spaces: An efficient algorithm based on immersion simulations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(6):583–598, 1991. doi: 10.1109/34.87344.

Phụ lục A

Phân công công việc

Cả ba thành viên cùng thực hiện phần lý thuyết, mô hình toán và tài liệu.

Thành viên	Công việc và artefact
Nguyễn Hồng Quang B23DCVT361	Package Python, test, CLI và `scripts/run_experiment.py`.
Nguyễn Trọng Nam Khánh B23DCCE052	Workflow Fiji/ImageJ: plugin StarDist, screenshot thao tác, macro, run log và label TIFF.
Vũ Dũng B23DCVT110	Tìm kiếm, thu thập và chuẩn bị dataset; tổ chức split và ground truth.

Phụ lục B

Hướng dẫn tái lập đầy đủ

B.1 Cài môi trường và kiểm thử

```
python -m venv .venv
python -m pip install -r requirements-model.txt
python -m pip install -e ".[dev]"
python -m pytest -q
python -m pytest -m integration -q
.venv\Scripts\ruff.exe check src tests scripts
```

B.2 Tạo lại kết quả và artefact

```
python scripts/run_dsb2018_benchmark.py
python scripts/run_validation_ablation.py
python scripts/evaluate_fiji_reference.py
python scripts/compare_fiji_python.py
python scripts/build_fiji_workflow_evidence.py
python scripts/build_report_data.py
```

Lệnh benchmark chính là ‘scripts/run_dsb2018_benchmark.py‘; dataset TIFF đã có trong repository, còn script chỉ tải archive DSB2018 và model ‘2D_versatile_fluo‘ qua API chính thức khi chưa có cache. Nó lưu CSV, label TIFF minh hoạ và figures. Lần chạy cập nhật ngày 24/10/2026 dùng CPU Intel macOS 15.7, Python 3.11.9, TensorFlow 2.15.1; TensorFlow native Windows từ 2.11 không dùng CUDA, nên benchmark GPU phải ghi môi trường WSL2/Linux riêng. Phần ImageJ/Fiji tham chiếu đã lưu label TIFF, figure, metric và run log trên ba ca.

Phụ lục C

Mã nguồn cốt lõi

C.1 Matching và metric instance

Listing C.1: ‘metrics.py‘: IoU matrix, Hungarian matching và các metric.

```
"""Quantitative metrics for 2D instance segmentation."""

from __future__ import annotations

from dataclasses import asdict, dataclass

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.optimize import linear_sum_assignment


@dataclass(frozen=True)
class InstanceMetrics:
    """Detection and segmentation statistics at one IoU threshold."""

    threshold: float
    true_objects: int
    predicted_objects: int
    true_positives: int
    false_positives: int
    false_negatives: int
    precision: float
    recall: float
    f1: float
    accuracy: float
```

```
    mean_matched_iou: float
    panoptic_quality: float
    foreground_dice: float
    foreground_iou: float
    count_absolute_error: int

    def to_dict(self) -> dict[str, float | int]:
        """Return a JSON-serializable representation."""

        return asdict(self)


def _validate_label_pair(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
) -> tuple[NDArray[np.integer], NDArray[np.integer]]:
    true = np.asarray(ground_truth)
    pred = np.asarray(prediction)
    if true.shape != pred.shape or true.ndim != 2:
        raise ValueError("Ground truth and prediction must be 2D
           arrays with identical shape.")
    for name, array in (("ground truth", true), ("prediction", pred))
       :
        if not np.issubdtype(array.dtype, np.integer) or np.any(array
            < 0):
            raise ValueError(f"{name} must contain non-negative
               integer labels.")
    return true, pred


def relabel_sequential(labels: ArrayLike) -> NDArray[np.int32]:
    """Map arbitrary positive label IDs to the sequence 1..N."""

    array = np.asarray(labels)
    output = np.zeros(array.shape, dtype=np.int32)
    for new_id, old_id in enumerate(np.unique(array[array > 0]),
       start=1):
        output[array == old_id] = new_id
    return output
```

```
def intersection_over_union_matrix(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
) -> NDArray[np.float64]:
    """Return pairwise IoU for all positive ground-truth/prediction
       instances."""

    true, pred = _validate_label_pair(ground_truth, prediction)
    true = relabel_sequential(true)
    pred = relabel_sequential(pred)
    n_true = int(true.max(initial=0))
    n_pred = int(pred.max(initial=0))
    if n_true == 0 or n_pred == 0:
        return np.zeros((n_true, n_pred), dtype=np.float64)

    flat_true = true.ravel()
    flat_pred = pred.ravel()
    overlap = np.zeros((n_true + 1, n_pred + 1), dtype=np.int64)
    np.add.at(overlap, (flat_true, flat_pred), 1)
    intersection = overlap[1:, 1:].astype(np.float64)
    true_area = overlap[1:, :].sum(axis=1, keepdims=True)
    pred_area = overlap[:, 1:].sum(axis=0, keepdims=True)
    union = true_area + pred_area - intersection
    return np.divide(intersection, union, out=np.zeros_like(
       intersection), where=union > 0)


def _safe_ratio(numerator: float, denominator: float) -> float:
    return float(numerator / denominator) if denominator else 0.0


def matched_label_pairs(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
    threshold: float = 0.5,
) -> list[tuple[int, int, float]]:
    """Return original label IDs and IoUs of the accepted one-to-one
       assignment."""
    if not 0 < threshold <= 1:
        raise ValueError("threshold must be in (0, 1].")
    true, pred = _validate_label_pair(ground_truth, prediction)
```

```
    iou = intersection_over_union_matrix(true, pred)
    if not min(iou.shape):
        return []
    cost = -(iou >= threshold).astype(float) - iou / (2 * min(iou.
        shape))
    ii, jj = linear_sum_assignment(cost)
    true_ids, pred_ids = np.unique(true[true > 0]), np.unique(pred[
        pred > 0])
    return [
        (int(true_ids[i]), int(pred_ids[j]), float(iou[i, j]))
        for i, j in zip(ii, jj, strict=True)
        if iou[i, j] >= threshold
    ]


def evaluate_instances(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
    threshold: float = 0.5,
) -> InstanceMetrics:
    """Evaluate a prediction using one-to-one IoU matching.

    The assignment maximizes the number of pairs at or above ``
        threshold`` and
    uses IoU as a tie-breaker, following the StarDist matching
        implementation.
    """

    if not 0 < threshold <= 1:
        raise ValueError("threshold must be in (0, 1].")
    true, pred = _validate_label_pair(ground_truth, prediction)
    n_true, n_pred = len(np.unique(true[true > 0])), len(np.unique(
        pred[pred > 0]))
    matched_ious = np.asarray([v for _, _, v in matched_label_pairs(
        true, pred, threshold)])

    true_positives = int(matched_ious.size)
    false_positives = n_pred - true_positives
    false_negatives = n_true - true_positives
    precision = _safe_ratio(true_positives, true_positives +
        false_positives)
```

```
    recall = _safe_ratio(true_positives, true_positives +
        false_negatives)
    f1 = _safe_ratio(2 * true_positives, 2 * true_positives +
        false_positives + false_negatives)
    accuracy = _safe_ratio(true_positives, true_positives +
        false_positives + false_negatives)
    mean_matched_iou = float(matched_ious.mean()) if true_positives
        else 0.0
    pq_denominator = true_positives + 0.5 * false_positives + 0.5 *
        false_negatives
    panoptic_quality = _safe_ratio(float(matched_ious.sum()),
        pq_denominator)

    true_foreground = true > 0
    pred_foreground = pred > 0
    intersection = int(np.logical_and(true_foreground,
        pred_foreground).sum())
    true_area = int(true_foreground.sum())
    pred_area = int(pred_foreground.sum())
    foreground_dice = _safe_ratio(2 * intersection, true_area +
        pred_area)
    foreground_iou = _safe_ratio(intersection, true_area + pred_area
        - intersection)

    return InstanceMetrics(
        threshold=float(threshold),
        true_objects=n_true,
        predicted_objects=n_pred,
        true_positives=true_positives,
        false_positives=false_positives,
        false_negatives=false_negatives,
        precision=precision,
        recall=recall,
        f1=f1,
        accuracy=accuracy,
        mean_matched_iou=mean_matched_iou,
        panoptic_quality=panoptic_quality,
        foreground_dice=foreground_dice,
        foreground_iou=foreground_iou,
        count_absolute_error=abs(n_pred - n_true),
    )
```

```
def evaluate_at_thresholds(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
    thresholds: list[float] | tuple[float, ...],
) -> list[InstanceMetrics]:
    """Evaluate the same label pair at multiple IoU thresholds."""

    if not thresholds:
        raise ValueError("At least one threshold is required.")
    return [evaluate_instances(ground_truth, prediction, value) for
       value in thresholds]
```

C.2 Star-convex geometry reference

Listing C.2: ‘geometry.py‘: EDT, ray distance, polygon decode và NMS.

```
"""Reference implementations of StarDist geometric building blocks.

These functions prioritize readability and verifiability over raw
   speed. They are
intended for coursework demonstrations and unit tests, not full model
    training.
"""

from __future__ import annotations

from collections.abc import Sequence

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy import ndimage as ndi
from skimage.draw import polygon as rasterize_polygon


def _validate_labels(labels: ArrayLike) -> NDArray[np.integer]:
    array = np.asarray(labels)
    if array.ndim != 2 or not np.issubdtype(array.dtype, np.integer):
        raise ValueError("Instance labels must be a 2D integer array.
           ")
```

```
    if np.any(array < 0):
        raise ValueError("Instance labels must be non-negative.")
    return array


def normalized_edt(labels: ArrayLike) -> NDArray[np.float32]:
    """Compute an Euclidean distance target normalized per object
       instance.

    Background is zero. Every positive object is transformed
       independently and
    divided by its own maximum. Image edges are not assumed to be
       background,
    matching StarDist's treatment of truncated objects. A full-frame
       instance
    is the exceptional case where background outside the image is
       assumed.
    """

    label_array = _validate_labels(labels)
    probability = np.zeros(label_array.shape, dtype=np.float32)
    for object_id in np.unique(label_array):
        if object_id == 0:
            continue
        mask = label_array == object_id
        if np.all(mask):
            padded = np.pad(mask, 1, mode="constant", constant_values
               =False)
            distance = ndi.distance_transform_edt(padded)[1:-1, 1:-1]
        else:
            distance = ndi.distance_transform_edt(mask)
        maximum = float(distance[mask].max(initial=0.0))
        if maximum > 0:
            probability[mask] = distance[mask] / maximum
    return probability


def ray_angles(n_rays: int) -> NDArray[np.float64]:
    """Return ``n_rays`` equally spaced angles in ``[0, 2*pi)``."""

    if n_rays < 3:
```

```
        raise ValueError("n_rays must be at least 3.")
    return np.linspace(0.0, 2.0 * np.pi, n_rays, endpoint=False)


def star_distances(
    labels: ArrayLike,
    n_rays: int = 32,
    step: float = 1.0,
) -> NDArray[np.float32]:
    """Estimate radial distances from every foreground pixel to its
       boundary.

    The implementation walks along equally spaced rays until the
       nearest sampled
    pixel no longer has the same instance ID. It is intentionally
       simple and slow,
    so use small masks to explain or validate the StarDist target
       representation.
    """

    label_array = _validate_labels(labels)
    if step <= 0:
        raise ValueError("step must be positive.")

    height, width = label_array.shape
    max_distance = float(np.hypot(height, width) + step)
    angles = ray_angles(n_rays)
    directions = np.column_stack((np.sin(angles), np.cos(angles)))
    distances = np.zeros((height, width, n_rays), dtype=np.float32)

    for y0, x0 in np.argwhere(label_array > 0):
        object_id = label_array[y0, x0]
        for ray_index, (dy, dx) in enumerate(directions):
            distance = step
            while distance <= max_distance:
                y = int(np.rint(y0 + distance * dy))
                x = int(np.rint(x0 + distance * dx))
                if y < 0 or y >= height or x < 0 or x >= width:
                    break
                if label_array[y, x] != object_id:
                    break
```

```
                distance += step
            distances[y0, x0, ray_index] = distance
    return distances


def decode_polygon(
    center_yx: Sequence[float],
    distances: ArrayLike,
) -> NDArray[np.float64]:
    """Decode radial distances into polygon vertices in ``(x, y)``
       order."""

    center = np.asarray(center_yx, dtype=np.float64)
    radii = np.asarray(distances, dtype=np.float64)
    if center.shape != (2,) or radii.ndim != 1 or radii.size < 3:
        raise ValueError("Expected center_yx=(y, x) and a 1D vector
           with at least 3 radii.")
    if np.any(radii < 0):
        raise ValueError("Polygon radii must be non-negative.")

    y0, x0 = center
    angles = ray_angles(radii.size)
    x = x0 + radii * np.cos(angles)
    y = y0 + radii * np.sin(angles)
    return np.column_stack((x, y))


def polygon_area(polygon_xy: ArrayLike) -> float:
    """Calculate polygon area with the shoelace formula."""

    polygon = np.asarray(polygon_xy, dtype=np.float64)
    if polygon.ndim != 2 or polygon.shape[1] != 2 or polygon.shape[0]
        < 3:
        raise ValueError("A polygon must have shape (N, 2) with N >=
           3.")
    x, y = polygon[:, 0], polygon[:, 1]
    return float(0.5 * abs(np.dot(x, np.roll(y, -1)) - np.dot(y, np.
       roll(x, -1))))


def polygon_overlap_ratio(
```

```
    polygon_a_xy: ArrayLike,
    polygon_b_xy: ArrayLike,
    mode: str = "smaller",
) -> float:
    """Estimate polygon overlap after rasterization on a shared local
        canvas.

    ``mode='smaller'`` returns intersection divided by the smaller
       polygon area,
    which mirrors StarDist 2D NMS. ``mode='iou'`` returns the
       standard IoU.
    """

    polygon_a = np.asarray(polygon_a_xy, dtype=np.float64)
    polygon_b = np.asarray(polygon_b_xy, dtype=np.float64)
    if mode not in {"smaller", "iou"}:
        raise ValueError("mode must be either 'smaller' or 'iou'.")
    for polygon in (polygon_a, polygon_b):
        if polygon.ndim != 2 or polygon.shape[1] != 2 or polygon.
           shape[0] < 3:
            raise ValueError("Every polygon must have shape (N, 2)
               with N >= 3.")

    all_points = np.vstack((polygon_a, polygon_b))
    min_x, min_y = np.floor(all_points.min(axis=0)).astype(int) - 1
    max_x, max_y = np.ceil(all_points.max(axis=0)).astype(int) + 1
    width = max_x - min_x + 1
    height = max_y - min_y + 1
    if width <= 0 or height <= 0:
        return 0.0

    masks: list[NDArray[np.bool_]] = []
    for polygon in (polygon_a, polygon_b):
        shifted_x = polygon[:, 0] - min_x
        shifted_y = polygon[:, 1] - min_y
        rows, cols = rasterize_polygon(shifted_y, shifted_x, shape=(
           height, width))
        mask = np.zeros((height, width), dtype=bool)
        mask[rows, cols] = True
        masks.append(mask)
```

```
    area_a = int(masks[0].sum())
    area_b = int(masks[1].sum())
    intersection = int(np.logical_and(masks[0], masks[1]).sum())
    if mode == "smaller":
        denominator = min(area_a, area_b)
    else:
        denominator = area_a + area_b - intersection
    return intersection / denominator if denominator else 0.0


def greedy_polygon_nms(
    polygons_xy: Sequence[ArrayLike],
    scores: ArrayLike,
    threshold: float = 0.3,
    overlap_mode: str = "smaller",
) -> NDArray[np.int64]:
    """Return indices retained by score-sorted greedy polygon NMS."""

    if not 0 <= threshold <= 1:
        raise ValueError("threshold must be in [0, 1].")
    score_array = np.asarray(scores, dtype=np.float64)
    if score_array.ndim != 1 or score_array.size != len(polygons_xy):
        raise ValueError("scores must contain one value for every
           polygon.")

    order = np.argsort(-score_array, kind="stable")
    kept: list[int] = []
    for candidate in order:
        if all(
            polygon_overlap_ratio(
                polygons_xy[int(candidate)], polygons_xy[selected],
                   mode=overlap_mode
            )
            <= threshold
            for selected in kept
        ):
            kept.append(int(candidate))
    return np.asarray(kept, dtype=np.int64)
```

C.3 Script thí nghiệm chạy thật

Listing C.3: ‘run_real_demo.py‘: materialize sample, inference, evaluation và figure.

```
"""Run a real, reproducible StarDist demonstration on its DSB2018
   sample.

The official StarDist package distributes this fluorescence image and
    instance
mask as a sample from the 2018 Data Science Bowl. This script
   materializes the
sample locally, runs the three methods, exports TIFF labels/CSV/
   figures, and
marks the output as a one-image demonstration rather than a benchmark
   .
"""

from __future__ import annotations

import csv
import json
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from stardist.data import test_image_nuclei_2d
from stardist.models import StarDist2D
from tifffile import imwrite

from xla_gr03.baselines import segment_otsu, segment_watershed
from xla_gr03.experiments import runtime_provenance
from xla_gr03.io import save_labels
from xla_gr03.metrics import evaluate_at_thresholds,
   evaluate_instances
from xla_gr03.preprocessing import percentile_normalize

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw" / "dsb2018_stardist_sample.tif"
GT = ROOT / "data" / "ground_truth" / "dsb2018_stardist_sample.tif"
OUT = ROOT / "results" / "real_demo"
LABELS = OUT / "labels"
```

```
FIGURES = OUT / "figures"
THRESHOLDS = (0.5, 0.75, 0.9)


def write_csv(path: Path, rows: list[dict[str, object]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    fields = list(dict.fromkeys(key for row in rows for key in row))
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields,
           extrasaction="ignore")
        writer.writeheader()
        writer.writerows(rows)


def save_panel(image: np.ndarray, truth: np.ndarray, predictions:
   dict[str, np.ndarray]) -> None:
    FIGURES.mkdir(parents=True, exist_ok=True)
    panels = [("Input fluorescence", image), ("Ground truth", truth),
        *predictions.items()]
    figure, axes = plt.subplots(1, len(panels), figsize=(4 * len(
       panels), 4))
    for axis, (name, data) in zip(axes, panels, strict=True):
        axis.imshow(data, cmap="gray" if name == "Input fluorescence"
            else "nipy_spectral")
        axis.set_title(name)
        axis.axis("off")
    figure.tight_layout()
    figure.savefig(FIGURES / "qualitative_comparison.png", dpi=200,
       bbox_inches="tight")
    plt.close(figure)


def save_sweep_plot(rows: list[dict[str, object]], value: str,
   filename: str) -> None:
    figure, axis = plt.subplots(figsize=(6, 4))
    for method, marker in (("probability_sweep", "o"), ("nms_sweep",
       "s")):
        subset = [row for row in rows if row["sweep"] == method]
        x_key = "probability_threshold" if method == "
           probability_sweep" else "nms_threshold"
        axis.plot(
```

```
            [float(row[x_key]) for row in subset],
            [float(row[value]) for row in subset],
            marker=marker,
            label=method.replace("_", " "),
        )
    axis.set_xlabel("Threshold")
    axis.set_ylabel(value.replace("_", " "))
    axis.grid(alpha=0.25)
    axis.legend()
    figure.tight_layout()
    figure.savefig(FIGURES / filename, dpi=200)
    plt.close(figure)


def predict(
    model: StarDist2D,
    image: np.ndarray,
    probability_threshold: float,
    nms_threshold: float,
) -> tuple[np.ndarray, float]:
    start = time.perf_counter()
    normalized = percentile_normalize(image, pmin=1.0, pmax=99.8,
       axes=(0, 1), clip=False)
    labels, _ = model.predict_instances(
        normalized,
        axes="YX",
        prob_thresh=probability_threshold,
        nms_thresh=nms_threshold,
    )
    return np.asarray(labels, dtype=np.int32), time.perf_counter() -
       start


def main() -> None:
    import tensorflow as tf

    tf.config.set_visible_devices([], "GPU")
    np.random.seed(42)
    image, truth = test_image_nuclei_2d(return_mask=True)
    RAW.parent.mkdir(parents=True, exist_ok=True)
    GT.parent.mkdir(parents=True, exist_ok=True)
```

```
OUT.mkdir(parents=True, exist_ok=True)
imwrite(RAW, image)
imwrite(GT, truth.astype(np.uint16))

methods: dict[str, tuple[np.ndarray, float, dict[str, object]]] =
    {}
for name, method in (
    ("otsu", lambda: segment_otsu(image, foreground="bright",
       min_size=20)),
    (
        "watershed",
        lambda: segment_watershed(
            image, foreground="bright", min_size=20, min_distance
               =7
        ),
    ),
):
    start = time.perf_counter()
    result = method()
    methods[name] = (result.labels, time.perf_counter() - start,
       result.metadata)

model = StarDist2D.from_pretrained("2D_versatile_fluo")
# Warm-up downloads/builds the model before the recorded
   inference.
predict(model, image, probability_threshold=0.5, nms_threshold
   =0.3)
labels, elapsed = predict(model, image, probability_threshold
   =0.5, nms_threshold=0.3)
methods["stardist_python"] = (
    labels,
    elapsed,
    {
        "model": "2D_versatile_fluo",
        "pmin": 1.0,
        "pmax": 99.8,
        "clip": False,
        "probability_threshold": 0.5,
        "nms_threshold": 0.3,
        "device": "CPU",
    },
```

```
)

rows: list[dict[str, object]] = []
panels: dict[str, np.ndarray] = {}
for name, (prediction, elapsed, metadata) in methods.items():
    save_labels(LABELS / f"{name}.tif", prediction)
    panels[name] = prediction
    for metric in evaluate_at_thresholds(truth, prediction,
       THRESHOLDS):
        rows.append(
            {
                "method": name,
                "runtime_s": round(elapsed, 6),
                "source": "official StarDist DSB2018 sample",
                **metadata,
                **metric.to_dict(),
            }
        )
write_csv(OUT / "metrics_per_threshold.csv", rows)
save_panel(image, truth, panels)

sweep_rows: list[dict[str, object]] = []
for probability_threshold in (0.3, 0.4, 0.5, 0.6, 0.7):
    prediction, elapsed = predict(model, image,
       probability_threshold, 0.3)
    metric = evaluate_instances(truth, prediction, threshold=0.5)
    sweep_rows.append(
        {
            "sweep": "probability_sweep",
            "probability_threshold": probability_threshold,
            "nms_threshold": 0.3,
            "runtime_s": round(elapsed, 6),
            **metric.to_dict(),
        }
    )
for nms_threshold in (0.1, 0.2, 0.3, 0.4, 0.5):
    prediction, elapsed = predict(model, image, 0.5,
       nms_threshold)
    metric = evaluate_instances(truth, prediction, threshold=0.5)
    sweep_rows.append(
        {
```

```
                "sweep": "nms_sweep",
                "probability_threshold": 0.5,
                "nms_threshold": nms_threshold,
                "runtime_s": round(elapsed, 6),
                **metric.to_dict(),
            }
        )
    write_csv(OUT / "stardist_threshold_sweep.csv", sweep_rows)
    save_sweep_plot(sweep_rows, "accuracy", "threshold_vs_ap050.png")
    save_sweep_plot(sweep_rows, "predicted_objects", "
       threshold_vs_object_count.png")
    (OUT / "provenance.json").write_text(
        json.dumps(
            {
                "dataset": "StarDist distributed DSB2018 fluorescence
                    sample",
                "source": "https://github.com/stardist/stardist",
                "sample_count": 1,
                "caveat": "Demonstration only. This single image is
                   not an independent test set.",
                "model": "2D_versatile_fluo",
                "random_seed": 42,
                "normalization_clip": False,
                "runtime_scope": "warm preprocessing+inference+
                   postprocessing; no I/O",
                "environment": runtime_provenance(model),
            },
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )


if __name__ == "__main__":
    main()
```

Listing C.4: ‘run_dsb2018_benchmark.py‘: validation, benchmark test split và tổng hợp kết quả.

```
"""Reproduce a multi-image DSB2018 benchmark for StarDist and two
   baselines.
```

```
The archive is the official split distributed by the StarDist project. The
training partition selects the marker spacing for Watershed; the separate test
partition reports the final metrics. The pretrained StarDist model runs with
its published default thresholds, without tuning on this benchmark's test set.
"""

from __future__ import annotations

import csv
import json
import statistics
import time
import urllib.request
import zipfile
from collections import defaultdict
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from stardist.models import StarDist2D
from tifffile import imread

from xla_gr03.baselines import segment_otsu, segment_watershed
from xla_gr03.experiments import dataset_manifest, paired_images, runtime_provenance
from xla_gr03.io import save_labels
from xla_gr03.metrics import evaluate_at_thresholds, evaluate_instances
from xla_gr03.preprocessing import percentile_normalize

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data" / "dsb2018"
ARCHIVE = DATA / "dsb2018.zip"
DATASET_ROOT = DATA / "dsb2018"
RESULTS = ROOT / "results" / "dsb2018_benchmark"
```

```
URL = "https://github.com/stardist/stardist/releases/download/0.1.0/
   dsb2018.zip"
IOU_THRESHOLDS = (0.5, 0.75, 0.9)
VALIDATION_MIN_DISTANCE = (3, 5, 7, 9, 11)


def write_csv(path: Path, rows: list[dict[str, object]]) -> None:
    fields = list(dict.fromkeys(key for row in rows for key in row))
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)


def ensure_dataset() -> None:
    test_dir = DATASET_ROOT / "test" / "images"
    if test_dir.is_dir() and any(test_dir.glob("*.tif")):
        return
    DATA.mkdir(parents=True, exist_ok=True)
    if not ARCHIVE.exists():
        print(f"Downloading {URL}")
        urllib.request.urlretrieve(URL, ARCHIVE)
    with zipfile.ZipFile(ARCHIVE) as archive:
        archive.extractall(DATA)
    if not test_dir.is_dir() or not any(test_dir.glob("*.tif")):
        raise RuntimeError("The official archive did not contain test
            image TIFF files.")


def paired_files(split: str) -> list[tuple[Path, Path]]:
    image_dir = DATASET_ROOT / split / "images"
    mask_dir = DATASET_ROOT / split / "masks"
    return paired_images(image_dir, mask_dir)


def choose_watershed_distance(
    validation: list[tuple[Path, Path]],
) -> tuple[int, list[dict[str, object]]]:
    rows: list[dict[str, object]] = []
    for distance in VALIDATION_MIN_DISTANCE:
        scores: list[float] = []
```

```
        for image_path, mask_path in validation:
            prediction = segment_watershed(
                imread(image_path), foreground="bright", min_size=20,
                    min_distance=distance
            ).labels
            scores.append(evaluate_instances(imread(mask_path),
               prediction, threshold=0.5).accuracy)
        rows.append(
            {
                "min_distance": distance,
                "n_validation": len(scores),
                "ap_iou_050_mean": float(statistics.mean(scores)),
                "ap_iou_050_sd": float(statistics.stdev(scores)) if
                   len(scores) > 1 else 0.0,
            }
        )
    best = max(rows, key=lambda row: (float(row["ap_iou_050_mean"]),
       -int(row["min_distance"])))
    return int(best["min_distance"]), rows


def predict_stardist(
    model: StarDist2D, image: np.ndarray
) -> tuple[np.ndarray, float, float, float]:
    probability_threshold = float(model.thresholds.prob)
    nms_threshold = float(model.thresholds.nms)
    start = time.perf_counter()
    normalized = percentile_normalize(image, pmin=1.0, pmax=99.8,
       axes=(0, 1), clip=False)
    labels, _ = model.predict_instances(
        normalized,
        axes="YX",
        prob_thresh=probability_threshold,
        nms_thresh=nms_threshold,
    )
    return (
        np.asarray(labels, dtype=np.int32),
        time.perf_counter() - start,
        probability_threshold,
        nms_threshold,
    )
```

```
def make_summary(rows: list[dict[str, object]]) -> list[dict[str,
    object]]:
    grouped: dict[tuple[str, float], list[dict[str, object]]] =
        defaultdict(list)
    for row in rows:
        grouped[(str(row["method"]), float(row["threshold"]))].append
            (row)
    summary: list[dict[str, object]] = []
    for (method, threshold), values in sorted(grouped.items()):
        numbers_by_metric = {
            key: [float(value[key]) for value in values]
            for key in (
                "accuracy",
                "panoptic_quality",
                "foreground_dice",
                "count_absolute_error",
                "runtime_s",
            )
        }

        def avg(key: str, numbers=numbers_by_metric) -> float:
            return float(statistics.mean(numbers[key]))

        def sd(key: str, numbers=numbers_by_metric) -> float:
            return float(statistics.stdev(numbers[key])) if len(
                numbers[key]) > 1 else 0.0

        summary.append(
            {
                "method": method,
                "threshold": threshold,
                "n_test": len(values),
                "accuracy_mean": avg("accuracy"),
                "accuracy_sd": sd("accuracy"),
                "panoptic_quality_mean": avg("panoptic_quality"),
                "panoptic_quality_sd": sd("panoptic_quality"),
                "foreground_dice_mean": avg("foreground_dice"),
                "foreground_dice_sd": sd("foreground_dice"),
                "count_absolute_error_mean": avg("
```

```
                    count_absolute_error"),
                "count_absolute_error_sd": sd("count_absolute_error")
                    ,
                "runtime_s_mean": avg("runtime_s"),
                "runtime_s_sd": sd("runtime_s"),
            }
        )
    return summary


def make_figure(summary: list[dict[str, object]]) -> None:
    rows_at_threshold = {
        str(row["method"]): row for row in summary if float(row["
            threshold"]) == 0.5
    }
    subset = [rows_at_threshold[method] for method in ("Otsu", "
        Watershed", "StarDist Python")]
    labels = [str(row["method"]) for row in subset]
    ap = [float(row["accuracy_mean"]) for row in subset]
    pq = [float(row["panoptic_quality_mean"]) for row in subset]
    x = np.arange(len(labels))
    figure, axis = plt.subplots(figsize=(7, 4.5))
    width = 0.36
    axis.bar(x - width / 2, ap, width, label="AP@0.50")
    axis.bar(x + width / 2, pq, width, label="PQ@0.50")
    axis.set_ylabel("Mean score on official test split")
    axis.set_ylim(0, 1)
    axis.set_xticks(x, labels)
    axis.legend()
    axis.grid(axis="y", alpha=0.25)
    figure.tight_layout()
    figure.savefig(RESULTS / "figures" / "benchmark_ap_pq.png", dpi
        =200)
    plt.close(figure)


def save_qualitative_panel(
    image_path: Path,
    min_distance: int,
    model: StarDist2D,
    tag: str,
```

```
) -> None:
    """Save raw/GT/predictions and TIFF labels for one selected test
       image."""

    image = imread(image_path)
    truth = imread(DATASET_ROOT / "test" / "masks" / image_path.name)
    otsu = segment_otsu(image, foreground="bright", min_size=20).
       labels
    watershed = segment_watershed(
        image, foreground="bright", min_size=20, min_distance=
           min_distance
    ).labels
    stardist, _, _, _ = predict_stardist(model, image)
    predictions = {
        "Otsu": otsu,
        "Watershed": watershed,
        "StarDist Python": stardist,
    }
    label_dir = RESULTS / "selected_labels" / tag
    for method, labels in predictions.items():
        save_labels(label_dir / f"{method.lower().replace(' ', '_')}.
           tif", labels)
    save_labels(label_dir / "ground_truth.tif", truth)
    figure, axes = plt.subplots(1, 5, figsize=(18, 4))
    panels = [("Input", image), ("Ground truth", truth), *predictions
       .items()]
    for axis, (title, array) in zip(axes, panels, strict=True):
        axis.imshow(array, cmap="gray" if title == "Input" else "
           nipy_spectral")
        axis.set_title(title)
        axis.axis("off")
    figure.suptitle(f"Fixed historical {tag} case (image IDs not
       reselected)", y=1.02)
    figure.tight_layout()
    figure.savefig(RESULTS / "figures" / f"qualitative_{tag}.png",
       dpi=200, bbox_inches="tight")
    plt.close(figure)


def main() -> None:
    tf.config.set_visible_devices([], "GPU")
```

```
ensure_dataset()
RESULTS.mkdir(parents=True, exist_ok=True)
(RESULTS / "figures").mkdir(exist_ok=True)
all_train = paired_files("train")
validation_indices = np.random.default_rng(42).choice(
    len(all_train), size=min(30, len(all_train)), replace=False
)
validation = [all_train[index] for index in sorted(
   validation_indices)]
test = paired_files("test")
manifest = dataset_manifest(all_train, "train") +
   dataset_manifest(test, "test")
train_hashes = {r["image_sha256"] for r in manifest if r["split"]
    == "train"}
test_hashes = {r["image_sha256"] for r in manifest if r["split"]
   == "test"}
if train_hashes & test_hashes:
    raise ValueError("Identical images occur in train and test
       splits.")
write_csv(RESULTS / "dataset_manifest.csv", manifest)
write_csv(RESULTS / "validation_images.csv", [{"image": p.name}
   for p, _ in validation])
selected_distance, validation_rows = choose_watershed_distance(
   validation)
write_csv(RESULTS / "watershed_validation_sweep.csv",
   validation_rows)

model = StarDist2D.from_pretrained("2D_versatile_fluo")
# Build the TensorFlow graph before recording inference times.
predict_stardist(model, imread(test[0][0]))
rows: list[dict[str, object]] = []
for image_path, mask_path in test:
    print(f"Benchmark {image_path.name}", flush=True)
    image = imread(image_path)
    truth = imread(mask_path)
    runs: list[tuple[str, np.ndarray, float, dict[str, object]]]
       = []
    start = time.perf_counter()
    otsu = segment_otsu(image, foreground="bright", min_size=20)
    runs.append(("Otsu", otsu.labels, time.perf_counter() - start
       , otsu.metadata))
```

```
    start = time.perf_counter()
    watershed = segment_watershed(
        image, foreground="bright", min_size=20, min_distance=
           selected_distance
    )
    runs.append(
        ("Watershed", watershed.labels, time.perf_counter() -
           start, watershed.metadata)
    )
    stardist, elapsed, probability_threshold, nms_threshold =
       predict_stardist(model, image)
    runs.append(
        (
            "StarDist Python",
            stardist,
            elapsed,
            {
                "model": "2D_versatile_fluo",
                "pmin": 1.0,
                "pmax": 99.8,
                "clip": False,
                "probability_threshold": probability_threshold,
                "nms_threshold": nms_threshold,
            },
        )
    )
    for method, prediction, elapsed, metadata in runs:
        for metric in evaluate_at_thresholds(truth, prediction,
           IOU_THRESHOLDS):
            rows.append(
                {
                    "image": image_path.name,
                    "method": method,
                    "runtime_s": elapsed,
                    **metadata,
                    **metric.to_dict(),
                }
            )
write_csv(RESULTS / "per_image_metrics.csv", rows)
summary = make_summary(rows)
write_csv(RESULTS / "summary_metrics.csv", summary)
```

```
make_figure(summary)
stardist_ap = sorted(
    (
        row
        for row in rows
        if row["method"] == "StarDist Python" and float(row["
           threshold"]) == 0.5
    ),
    key=lambda row: float(row["accuracy"]),
)
# Keep the historical Fiji cases fixed so reruns never break
   their paired reference.
historical = ROOT / "fiji" / "run_log_dsb2018_reference.csv"
with historical.open(newline="", encoding="utf-8") as stream:
    fixed_cases = list(csv.DictReader(stream))
image_index = {path.name: path for path, _ in test}
selected_rows: list[dict[str, object]] = []
for case in fixed_cases:
    tag = case["tag"]
    row = next(r for r in stardist_ap if r["image"] == case["
       image"])
    image_path = image_index[str(row["image"])]
    save_qualitative_panel(image_path, selected_distance, model,
       tag)
    selected_rows.append(
        {
            "tag": tag,
            "image": image_path.name,
            "stardist_ap_iou_050": row["accuracy"],
            "selection_rule": (
                "fixed historical min/median/max from original
                   clipped run; not reselected"
            ),
        }
    )
write_csv(RESULTS / "selected_cases.csv", selected_rows)
(RESULTS / "provenance.json").write_text(
    json.dumps(
        {
            "dataset": "Official StarDist DSB2018 archive",
            "archive_url": URL,
```

```
                "train_images_for_validation": len(validation),
                "test_images": len(test),
                "watershed_min_distance_selected_on_train":
                   selected_distance,
                "stardist_model": "2D_versatile_fluo",
                "stardist_threshold_policy": "model default
                   thresholds; no test-set tuning",
                "device": "CPU",
                "runtime_scope": "normalization+inference+
                   postprocessing; excludes I/O/load/warmup",
                "normalization": {
                    "pmin": 1.0,
                    "pmax": 99.8,
                    "clip": False,
                    "percentile": "numpy linear",
                    "eps": 1e-20,
                },
                "pretrained_test_independence": "not verified against
                    weight training IDs",
                "qualitative_selection": "fixed historical cases; no
                   reselection after rerun",
                "environment": runtime_provenance(model),
            },
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )


if __name__ == "__main__":
    main()
```

C.4 Runner StarDist ImageJ/Fiji

Listing C.5: `FijiStarDistRunner.java`: gọi command plugin, xuất Label Image TIFF và log runtime.

```
/**
```

```
 * Headless, reproducible runner for the published ImageJ/Fiji
    StarDist 2D command.
 *
 * <p>Usage: {@code FijiStarDistRunner input.tif label-output.tif [
    nTiles]}. The
 * StarDist 0.3.0-scijava bundled fluorescent-nuclei model is used
    with its
 * optimized thresholds: probability/score=0.479071 and overlap-NMS
    =0.3.</p>
 */

import java.util.HashMap;
import java.util.Map;

import de.csbdresden.stardist.StarDist2D;
import net.imagej.Dataset;
import net.imagej.ImageJ;
import org.scijava.command.CommandModule;

public final class FijiStarDistRunner {
    private FijiStarDistRunner() {}

    public static void main(final String... args) throws Exception {
        if (args.length < 2 || args.length > 3) {
            throw new IllegalArgumentException(
                "Usage: FijiStarDistRunner input.tif label-output.tif
                    [nTiles]"
            );
        }

        final int nTiles = args.length == 3 ? Integer.parseInt(args
           [2]) : 1;
        if (nTiles < 1) throw new IllegalArgumentException("nTiles
           must be >= 1");

        final ImageJ ij = new ImageJ();
        final Map<String, Object> parameters = new HashMap<>();
        parameters.put("input", (Dataset) ij.io().open(args[0]));
        parameters.put("modelChoice", "Versatile (fluorescent nuclei)
           ");
        parameters.put("normalizeInput", true);
```

```
        parameters.put("percentileBottom", 1.0);
        parameters.put("percentileTop", 99.8);
        parameters.put("probThresh", 0.479071);
        parameters.put("nmsThresh", 0.3);
        parameters.put("outputType", "Label Image");
        parameters.put("nTiles", nTiles);
        parameters.put("excludeBoundary", 2);
        parameters.put("roiPosition", "Automatic");
        parameters.put("verbose", false);
        parameters.put("showCsbdeepProgress", false);
        parameters.put("showProbAndDist", false);

        try {
            final long startNanos = System.nanoTime();
            final CommandModule module = ij.command().run(StarDist2D.
               class, false, parameters).get();
            final double elapsedSeconds = (System.nanoTime() -
               startNanos) / 1_000_000_000.0;
            final Dataset label = (Dataset) module.getOutput("label")
               ;
            if (label == null) throw new IllegalStateException("
               StarDist produced no Label Image output.");
            ij.io().save(label, args[1]);

            System.out.printf("INPUT=%s%nOUTPUT=%s%nMODEL=Versatile (
               fluorescent nuclei)%n" +
                "PROB_THRESH=0.479071%nNMS_THRESH=0.3%nNTILES=%d%
                   nRUNTIME_S=%.6f%n",
                args[0], args[1], nTiles, elapsedSeconds);
        } finally {
            ij.context().dispose();
        }
        System.exit(0);
    }
}
```